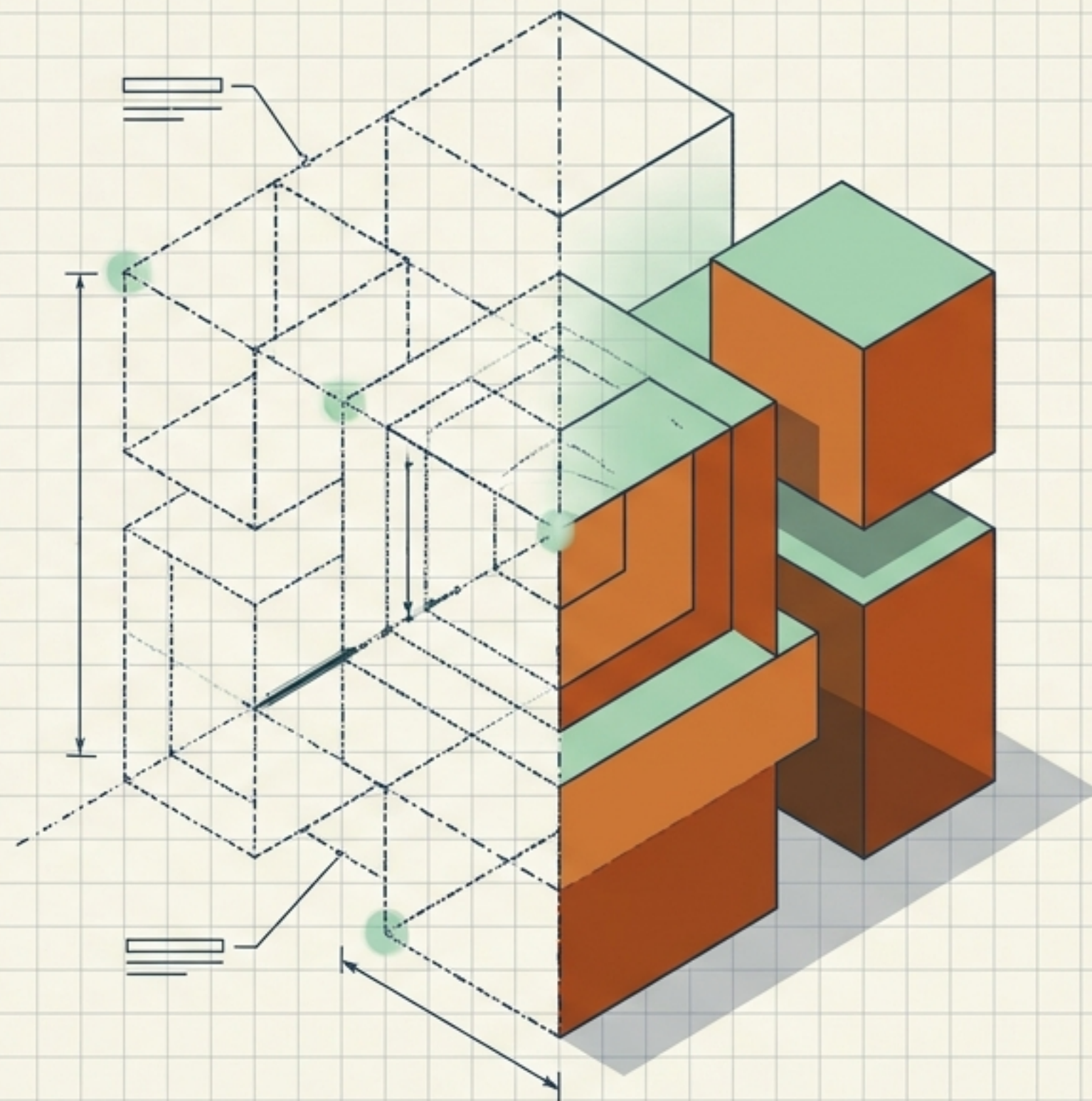
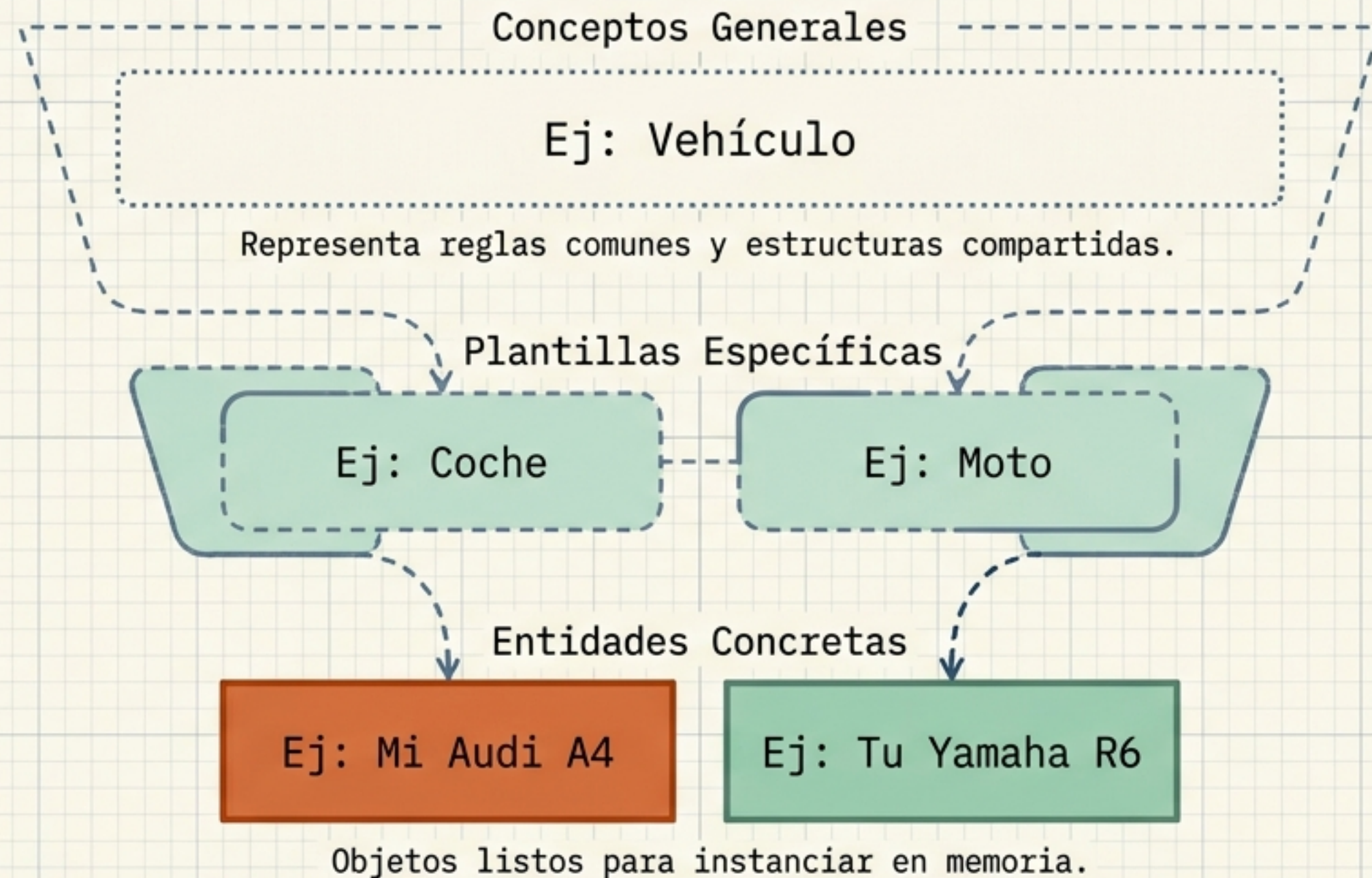


El Plano de la Abstracción

Arquitectura de Clases,
Interfaces y Contratos en Java



El Poder de la Abstracción



La abstracción separa el **QUÉ** debe hacer un objeto del **CÓMO** lo hace, permitiendo reutilizar código y estandarizar comportamientos mediante el **polimorfismo**.

Clases Abstractas: El Molde Incompleto

abstract class Mamifero



~~new Mamifero();~~

No instanciable. Solo define la base estructural.

```
Código
abstract class Mamifero {
    private String nombre;
    private int edad;

    public Mamifero(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

class Perro



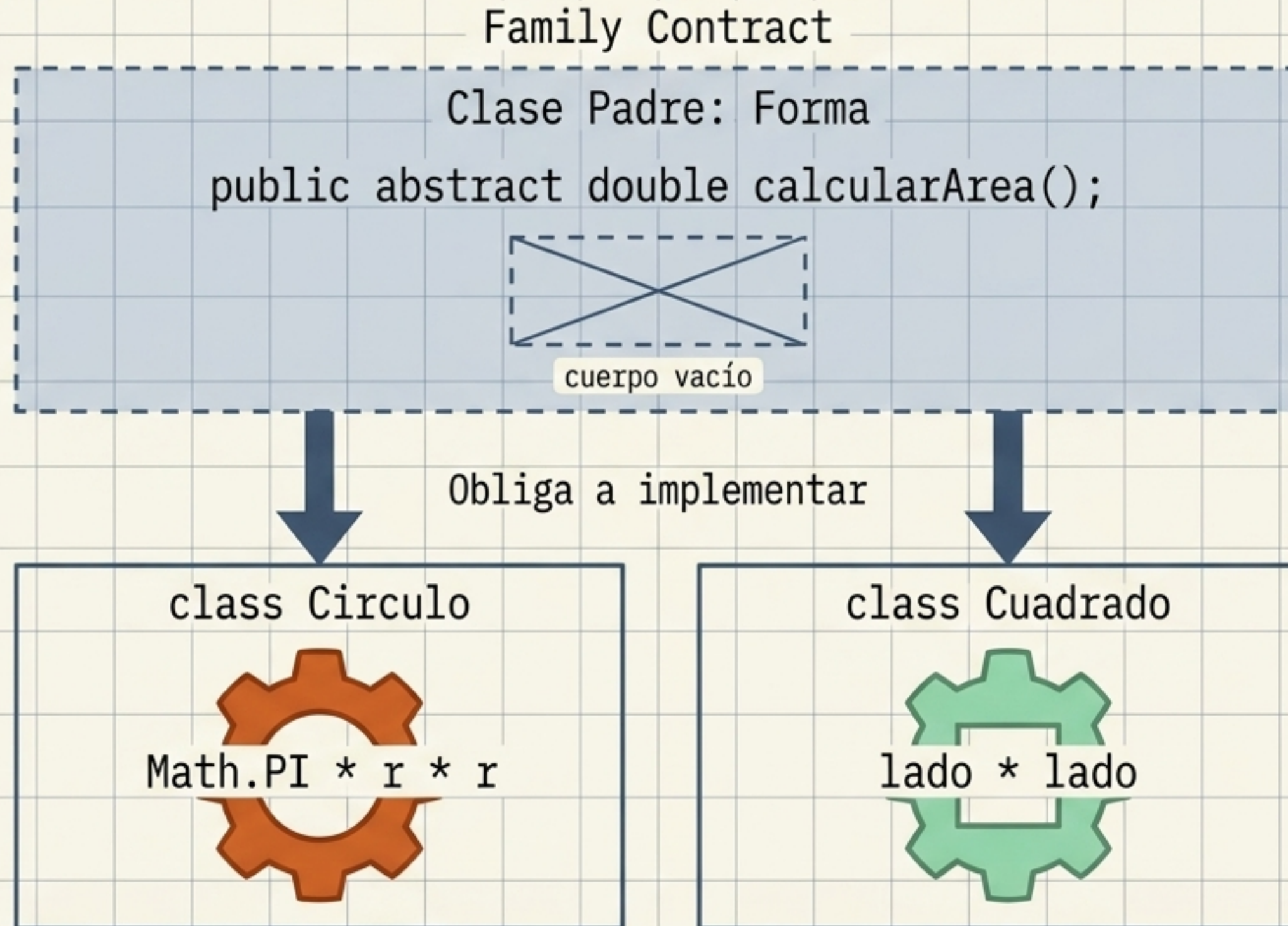
class Gato



new Perro();

Heredan el molde y completan la funcionalidad.

Métodos Abstractos: La Obligación del Heredero



La clase padre dicta la firma (el qué). La clase hija provee el cuerpo (el cómo).

Caso Práctico: El Linaje Musical

abstract class Instrumento

Nota[] melodia (Estado compartido)

add(Nota n) (Lógica compartida)

abstract void interpretar(); (Lógica faltante)



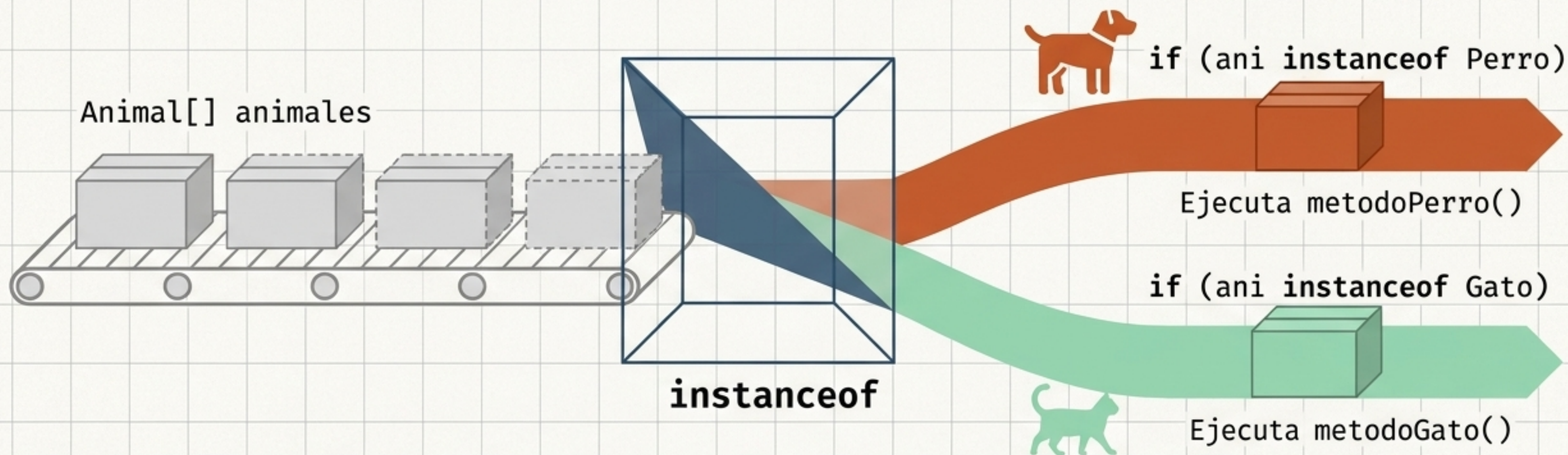
class Piano extends Instrumento

@Override void interpretar()

```
switch (nota) {  
  case DO: print('do');  
  case RE: print('re');  
  case MI: print('mi');  
  ...  
}
```

Instrumento
gestiona las notas almacenadas;
Piano decide cómo suenan al ejecutarse.

Selección Dinámica: El Escáner **instanceof**

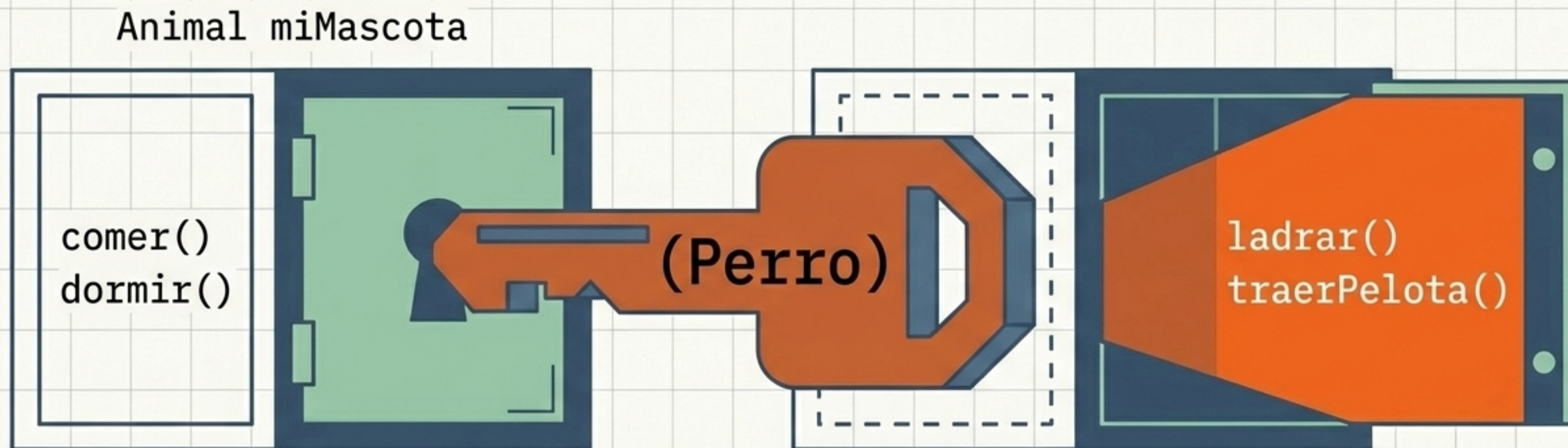


```
for(Animal ani : animales) {  
    if( ani instanceof Perro ) { Perro p = (Perro)ani; p.metodoPerro(); }  
    if( ani instanceof Gato ) { Gato p = (Gato)ani; p.metodoGato(); }  
}
```



Precaución: Es útil, pero un exceso de instanceof suele indicar que utilizar una Interfaz sería una mejor solución arquitectónica.

Casting de Objetos: Desbloqueando Capacidades



- **Problema:** Una referencia genérica (clase padre) oculta los métodos exclusivos de la subclase.
- **Solución:** El casting '`Perro p = (Perro) miMascota;`' transforma la lente, revelando la funcionalidad completa, siempre que el objeto real sea compatible.

Interfaces: El Contrato Universal

Limitación Genética

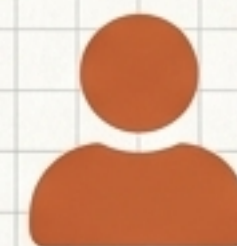
Clase Principal A

Subclase B

Subclase C

Un objeto solo puede heredar de una familia principal.

Interfaces: Flexibilidad Contractual

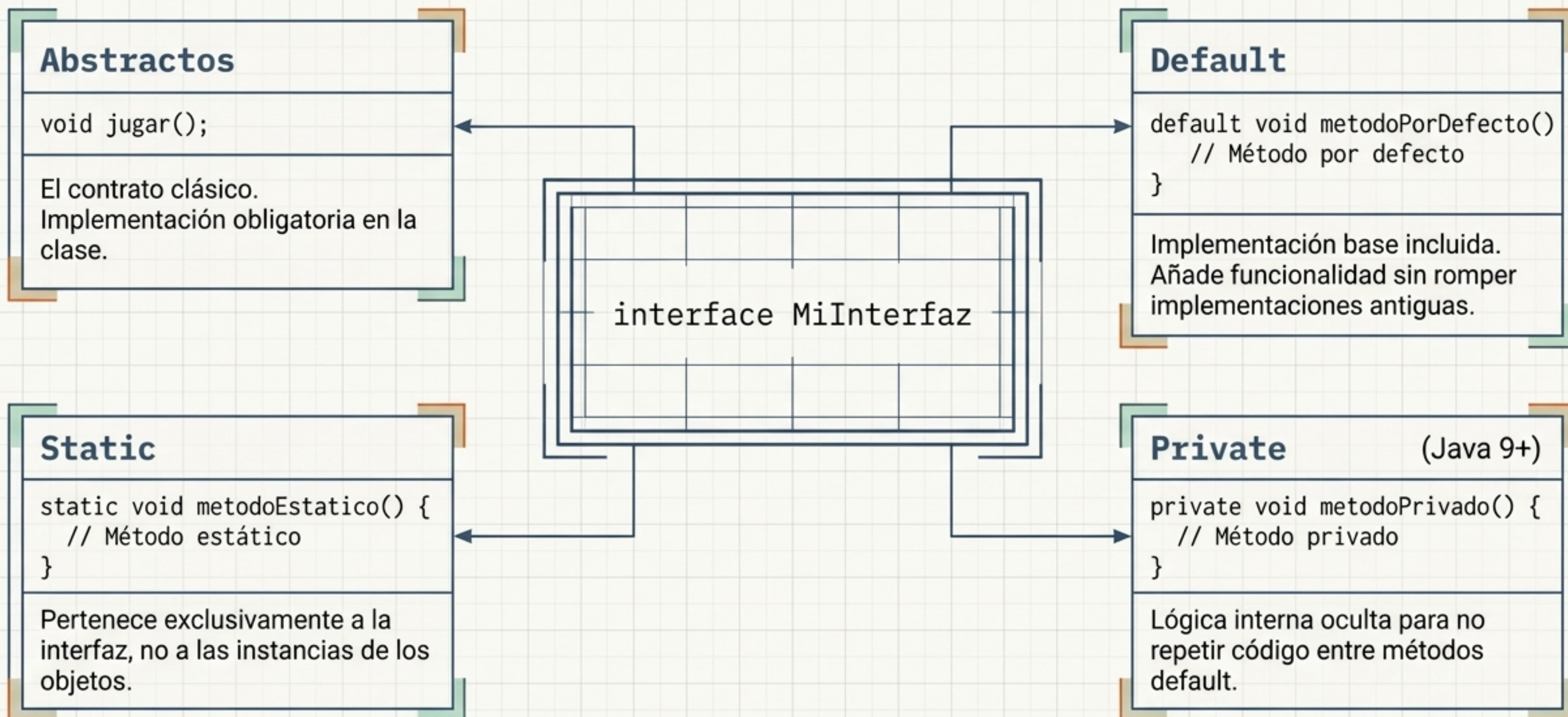


IReproductor

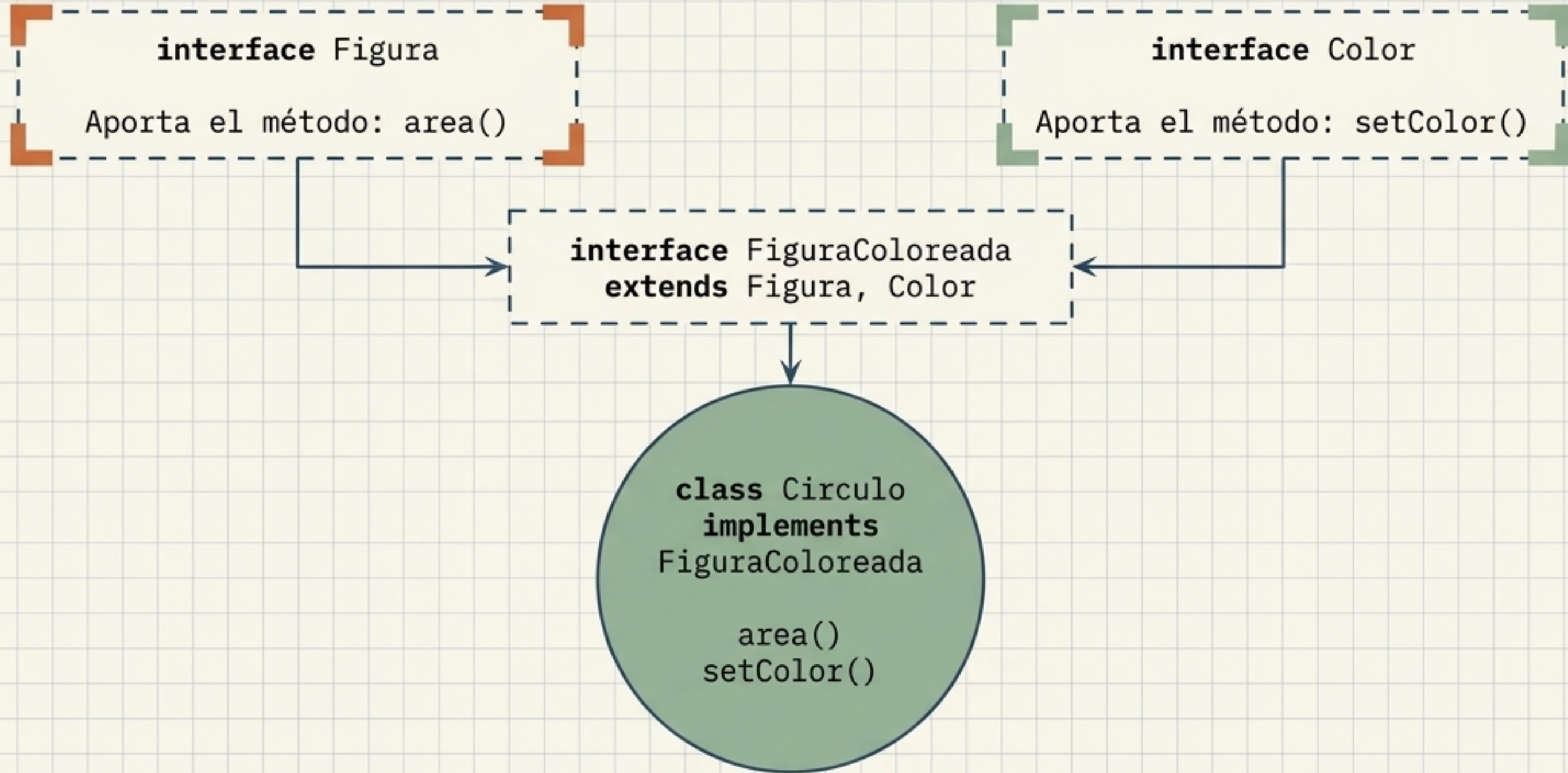
play()
pause()
stop()

Una **interfaz** es un **contrato de comportamiento**. Define QUÉ puede hacer un objeto garantizando estandarización, sin importar a qué familia jerárquica pertenezca.

Anatomía de una Interfaz Moderna



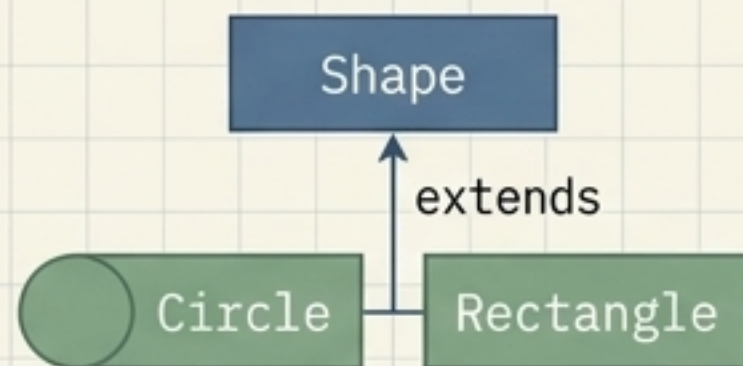
Simulando la Herencia Múltiple



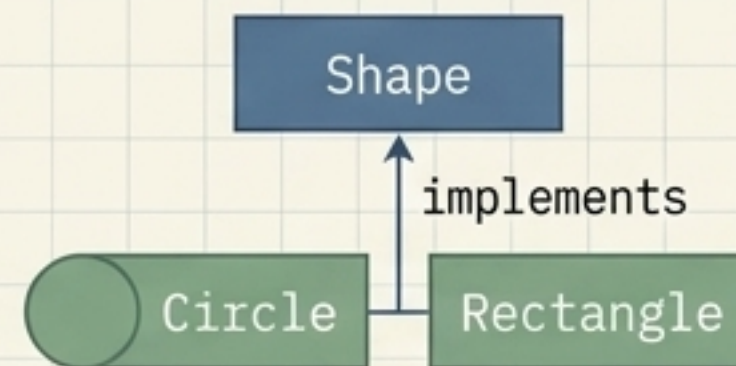
Aunque Java prohíbe que una clase herede de múltiples clases padre, permite implementar **infinitas interfaces simultáneamente**, logrando fusionar capacidades transversales.

El Enfrentamiento: Clase Abstracta vs. Interfaz

Clase Abstracta



Interfaz



Métodos

Abstractos y concretos.

Abstractos, default, static, private.

Herencia

Solo extiende 1 clase.

Implementa múltiples interfaces.

Estado
(Atributos)

✓ Variables de instancia.

✗ Solo constantes (final static).

Constructores

✓ Permitidos.

✗ Prohibidos.

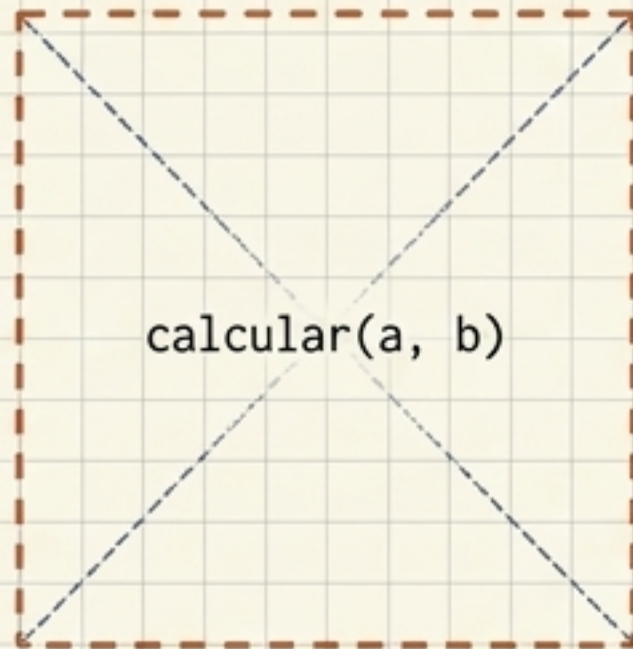
Caso de Uso
(Filosofía)

Identidad central ("Es un...").

Capacidades transversales
("Puede hacer...").

Clases Anónimas: Implementaciones Al Vuelo

interface Calculadora



Instanciación y Declaración

```
new Calculadora() {  
    @Override  
    public double calcular(double a, double b) {  
        return a + b;  
    }  
}
```

Ejecución

suma.calcular(5, 3) -> 8

El objeto nace, se define y se instancia sin crear un archivo de clase formal.

Útiles para crear implementaciones de interfaces de manera concisa, en un único lugar, ideales para usos de un solo objetivo.

Optimizando el Orden (API Java)

Comparable<T> (Orden Natural)



Hook: class Persona implements Comparable

Core Method: compareTo(T obj)

Define el orden por defecto inherente al objeto (ej. ordenar Personas por DNI). Modifica la clase original.

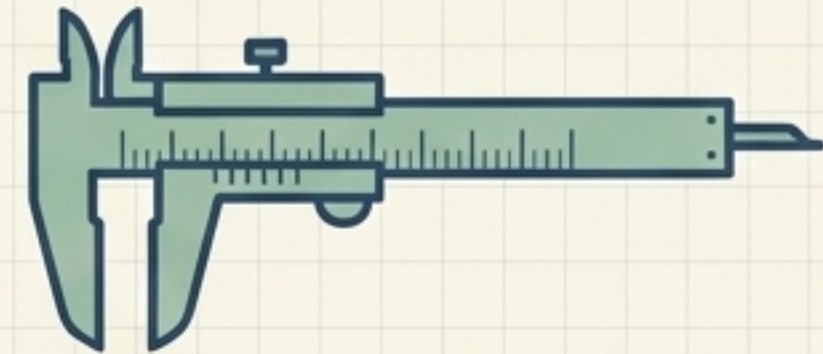
```
public class Persona implements Comparable<Persona> {
    private String nombre;
    private int edad;

    // Constructor, getters y setters

    @Override
    public int compareTo(Persona otraPersona) {
        return Integer.compare(this.edad, otraPersona.edad);
    }
}
```

Implementación de ordenamiento interno donde la clase compara su propio estado con otro objeto.

Comparator<T> (Orden Personalizado)



Hook: new Comparator<Persona>()

Core Metodo: compare(T obj1, T obj2)

Crea múltiples criterios de ordenación externos (ej. por edad, por nombre) sin alterar el código de la clase original.

```
import java.util.Comparator;

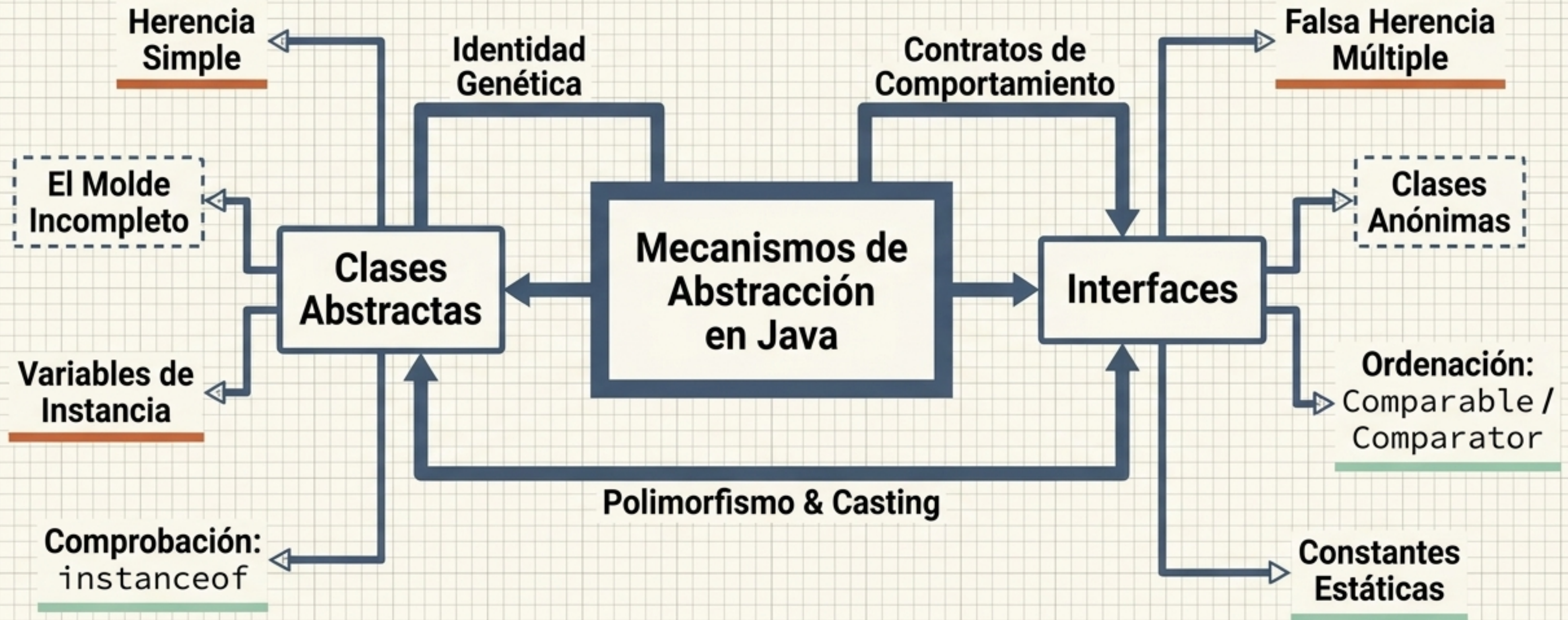
public class Persona {
    private String nombre;
    private int edad;

    // Constructor, getters y setters

    // Método estático que devuelve un Comparator para ordenar por edad de forma descendente
    public static Comparator<Persona> edadComparatorDescendente = new Comparator<Persona>() {
        @Override
        public int compare(Persona persona1, Persona persona2) {
            return Integer.compare(persona2.edad, persona1.edad); // Orden descendente
        }
    };
}
```

Implementación de ordenamiento externo donde una lógica estática maneja dos objetos separados.

El Mapa de la Abstracción



"Las clases abstractas definen QUÉ ES una entidad; las interfaces definen CÓMO SE COMPORTA."